
A. JOURNAUX SYSTÈME – SYSTEMD / JOURNALD

```
[USER] journalctl
```

```
DESCRIPTION : Consulte les journaux du système gérés par systemd-journal
POURQUOI : Accès centralisé à tous les journaux : noyau, services, applications,
          boot. Remplace la lecture manuelle de /var/log/* sur les systèmes systemd.
CONTEXTE : Diagnostic système, débogage de services, audit, surveillance.
OPTIONS :
  -f          Suit le journal en temps réel (follow – comme tail -f)
  -e          Sautte directement à la fin du journal
  -n N        Affiche les N dernières lignes
  -r          Ordre inverse (plus récent en premier)
  -u SERVICE  Filtre par unité systemd
  -p PRIORITÉ Filtre par priorité (0=emerg à 7=debug, ou noms)
  -k          Messages du noyau uniquement (comme dmesg)
  -b N        Messages depuis le Nième boot (0=actuel, -1=précédent)
  --list-boots Liste tous les boots enregistrés
  --since "DATE" Depuis cette date ("2024-01-01", "yesterday", "1 hour ago")
  --until "DATE" Jusqu'à cette date
  -o FORMAT   Format de sortie (short, verbose, json, json-pretty, cat, export)
  --no-pager   Pas de pagination (pour scripts)
  -x          Ajoute des explications contextuelles
  -g MOTIF     Filtre par expression régulière dans le message
  _PID=PID     Filtre par PID
  _UID=UID     Filtre par UID
  _SYSTEMD_UNIT=SERVICE Filtre par unité
  _COMM=NOM    Filtre par nom de commande
  --disk-usage Affiche l'espace disque utilisé par les journaux
  --vacuum-size=TAILLE Réduit les journaux à cette taille max
  --vacuum-time=DURÉE  Supprime les journaux plus vieux que cette durée
  --rotate      Force la rotation des journaux
  --verify      Vérifie l'intégrité des journaux
  -D RÉPERTOIRE Lit les journaux depuis ce répertoire
  -m           Fusionne les journaux de tous les namespaces
  -a           Affiche tous les champs (y compris binaires)
  --catalog     Affiche les entrées du catalogue de messages
  -t IDENTIFIANT Filtre par syslog identifier
  -q           Mode silencieux (pas de messages d'info)
PRIORITÉS :
  0 emerg      1 alert      2 crit      3 err
  4 warning    5 notice     6 info      7 debug
EXEMPLES :
$ journalctl -f
$ journalctl -u nginx -f
$ journalctl -u nginx --since "1 hour ago"
$ journalctl -p err -b
$ journalctl -k -b -1
$ journalctl --since "2024-01-15" --until "2024-01-16"
$ journalctl -o json-pretty -u sshd | head -50
$ journalctl --disk-usage
$ sudo journalctl --vacuum-time=30d
$ journalctl _PID=1234
$ journalctl -b --list-boots
$ journalctl -g "error|failed" -u postgresql
```

```
[USER] logger
```

```
DESCRIPTION : Envoie des messages vers le journal système (syslog/journald)
POURQUOI : Écrire des messages de log depuis des scripts shell ou en ligne de
          commande – tout converge vers journald/syslog.
CONTEXTE : Scripts shell, tests de journalisation, intégration d'applications.
OPTIONS :
  -p PRIORITÉ Priorité (facility.level, ex: user.info, daemon.err)
  -t TAG       Tag/identifiant du message
  -f FICHIER   Lit les messages depuis un fichier
```

```

-n HOTE          Envoie vers ce serveur syslog distant
-P PORT          Port du serveur syslog
-T              Utilise TCP (défaut UDP pour distant)
-u SOCKET        Utilise ce socket Unix
-i              Ajoute le PID au tag
-s              Affiche aussi sur stderr
--id=PID         Spécifie un ID de message
--msgid=ID       Identifiant RFC 5424
--journald       Envoie vers journald avec champs structurés
FACILITÉS       : kern, user, mail, daemon, auth, syslog, lpr, news, cron, local0-7
NIVEAUX         : emerg, alert, crit, err, warning, notice, info, debug
EXEMPLES        :
$ logger "Script de sauvegarde terminé"
$ logger -p daemon.err -t monapp "Erreur critique détectée"
$ logger -t backup -p local0.info "Backup réussi : $(du -sh /backup)"
$ logger -n syslog.entreprise.com -p user.info "Événement envoyé au SIEM"
$ echo "Mon message" | logger -t monscript

```

```
[USER] systemd-cat
```

```

DESCRIPTION : Connecte la sortie d'une commande au journal systemd (journald)
POURQUOI    : Alternative à logger pour journald – capture stdout et stderr
               d'une commande et les envoie directement dans le journal.
CONTEXTE    : Intégration de scripts dans journald, logging d'applications.
OPTIONS      :
  -t IDENTIFIANT      Identifiant syslog (tag)
  -p PRIORITÉ         Priorité pour stdout
  --stderr-priority=PRIO Priorité pour stderr
  --level-prefix=BOOL Interprète les préfixes de niveau <N>
EXEMPLES       :
$ systemd-cat echo "Message dans le journal"
$ systemd-cat -t monscript -p info ./mon_script.sh
$ echo "test" | systemd-cat -t debug_app
$ systemd-cat -t backup tar czf /backup/home.tar.gz /home/

```

```
[USER] dmesg
```

```

DESCRIPTION : Affiche le tampon de messages du noyau (kernel ring buffer)
POURQUOI    : Voir les messages du noyau : détection du matériel au boot, erreurs
               de drivers, problèmes hardware, messages des modules.
CONTEXTE    : Diagnostic matériel, débogage de drivers, analyse de boot.
OPTIONS      :
  -H          Format lisible avec couleurs et timestamps relatifs
  -T          Timestamps lisibles (date/heure réelles)
  -w          Suit les nouveaux messages en temps réel (watch)
  -f FACILITÉ Filtre par facilité (kern, user, daemon...)
  -l NIVEAU   Filtre par niveau (err, warn, info, debug...)
  -n NIVEAU   Définit le niveau de console
  -c          Efface le buffer après affichage (SUDO)
  --since "HORODATAGE" Depuis cette date
  --until "HORODATAGE" Jusqu'à cette date
  -e          Saute à la fin
  -k          Messages du noyau uniquement
  -u          Messages de l'espace utilisateur
  -x          Affiche les facilités et niveaux en texte
  -r          Inclut le niveau brut
  -s TAILLE   Taille du buffer de lecture
  --color=auto Coloration selon le niveau
  -P          Affiche les résumés de panic
EXEMPLES     :
$ dmesg -H
$ dmesg -T | grep -i error
$ dmesg -w
$ dmesg -l err,crit,emerg
$ dmesg | grep -i "usb\sata\nvme"
$ dmesg --since "10 minutes ago"
$ sudo dmesg -c > /tmp/dmesg_avant_test.txt

```

B. ANALYSE DE DÉMARRAGE & SYSTÈME

```
[USER] systemd-analyze
```

```
DESCRIPTION : Analyse le temps de démarrage systemd et les performances du boot
POURQUOI    : Identifier les services qui ralentissent le démarrage, optimiser le
              temps de boot, analyser les dépendances entre unités.
CONTEXTE    : Optimisation du boot, diagnostic de démarrage lent, analyse systemd.
OPTIONS     :
  (sans argument)      Temps total de boot (firmware + loader + kernel + userspace)
  blame                Liste les unités triées par temps de démarrage
  critical-chain        Affiche la chaîne critique qui détermine le temps de boot
  plot                 Génère un SVG du timeline de boot
  dot                  Génère un graphe DOT des dépendances
  dump                 Dump complet de l'état systemd
  cat-config FICHIER   Affiche la config effective d'une unité
  verify FICHIER       Vérifie la syntaxe d'une unité
  security UNITÉ       Analyse le score de sécurité d'une unité
  syscall-filter       Liste les appels système filtrés
  calendar "EXPR"      Analyse une expression de calendrier systemd
  timespan "EXPR"      Analyse un timespan systemd
  condition UNITÉ      Évalue les conditions d'une unité
EXEMPLES      :
$ systemd-analyze
$ systemd-analyze blame | head -20
$ systemd-analyze critical-chain
$ systemd-analyze plot > /tmp/boot.svg
$ systemd-analyze security nginx.service
$ systemd-analyze verify /etc/systemd/system/monservice.service
$ systemd-analyze calendar "Mon *-.* 09:00:00"
```

```
[USER] uname
```

```
DESCRIPTION : Affiche les informations sur le noyau et le système
POURQUOI    : Identifier rapidement la version du noyau, l'architecture, le hostname.
CONTEXTE    : Scripts de compatibilité, diagnostic, vérification d'architecture.
OPTIONS     :
  -a                Toutes les informations
  -r                Version du noyau (release)
  -s                Nom du noyau (Linux)
  -n                Hostname réseau
  -m                Architecture matérielle (x86_64, aarch64...)
  -p                Type de processeur
  -i                Plateforme matérielle
  -o                Système d'exploitation
  -v                Version (date de compilation du noyau)
EXEMPLES      :
$ uname -a
$ uname -r
$ uname -m
$ uname -mrs
```

```
[USER] hostname / hostnamectl
```

```
DESCRIPTION : hostname : affiche ou définit le nom d'hôte
              hostnamectl : gestion avancée du hostname via systemd
POURQUOI    : Identifier et configurer le nom d'hôte du système.
CONTEXTE    : Configuration système, scripts, identification de machines.
OPTIONS hostname :
  -f / --fqdn       Nom de domaine complet (FQDN)
  -i / --ip-address Adresses IP correspondantes
  -d / --domain     Nom de domaine
  -s / --short      Nom court
OPTIONS hostnamectl :
  status            Affiche toutes les infos hostname
  set-hostname NOM  Définit le hostname statique
  set-hostname NOM --transient Hostname temporaire (perdu au reboot)
  set-hostname NOM --pretty Nom d'affichage (peut contenir espaces/unicode)
  set-icon-name NOM  Icône (computer, server, laptop...)
  set-chassis TYPE   Type de chassis (desktop, laptop, server...)
  set-deployment ENV Environnement (development, staging, production)
  set-location LIEU  Localisation physique
EXEMPLES      :
$ hostname
$ hostname -f
```

```
$ hostnamectl status
$ sudo hostnamectl set-hostname serveur-prod-01
$ sudo hostnamectl set-deployment production
```

```
[USER] lscpu
```

```
DESCRIPTION : Affiche des informations détaillées sur le(s) processeur(s)
POURQUOI    : Connaître l'architecture CPU, le nombre de cœurs, les threads, le cache,
              les extensions supportées (AVX, AES, VMX/SVM pour la virtualisation).
CONTEXTE    : Compatibilité logicielle, optimisation, diagnostic de virtualisation.
OPTIONS     :
  -a          Toutes les CPUs (online + offline)
  -e          Format étendu (tableau par CPU)
  -p          Format parseable (pour scripts)
  -s RÉPERTOIRE Lit depuis un répertoire système (ex: /sys)
  -J          Sortie JSON
  -x          Affiche les données hexadécimales
EXEMPLES    :
$ lscpu
$ lscpu -J
$ lscpu | grep -E "^CPU\(s\)|Thread|Core|Socket|Model name"
$ lscpu | grep -i virtualization
```

```
[USER] lsmem
```

```
DESCRIPTION : Affiche les plages de mémoire RAM du système
POURQUOI    : Voir la disposition physique de la mémoire, les bancs de RAM,
              la mémoire online/offline, les zones NUMA.
CONTEXTE    : Diagnostic mémoire, configuration NUMA, administration serveur.
OPTIONS     :
  -a          Toutes les plages (online + offline)
  -o          Mémoire online uniquement
  -p          Format parseable
  --summary   Affiche un résumé
  -J          Sortie JSON
  --split=COLONNES Divise par ces colonnes
EXEMPLES    :
$ lsmem
$ lsmem --summary
$ lsmem -J
```

```
[USER] lsirq
```

```
DESCRIPTION : Affiche les interruptions matérielles et leur distribution par CPU
POURQUOI    : Identifier les interruptions et leur charge sur les CPUs – utile pour
              l'optimisation des performances réseau/disque (IRQ balancing).
CONTEXTE    : Optimisation des performances, diagnostic d'interruptions, NUMA.
OPTIONS     :
  -s          Trie par nombre total d'interruptions
  -c          Affiche les colonnes par CPU
EXEMPLES    :
$ lsirq
$ lsirq -s | head -20
$ cat /proc/interrupts
```

```
[SUDO] dmidecode
```

```
DESCRIPTION : Décode et affiche les informations DMI/SMBIOS du BIOS
POURQUOI    : Obtenir des informations matérielles précises directement depuis le BIOS :
              modèle de machine, numéro de série, RAM physique, slots, BIOS version.
CONTEXTE    : Inventaire matériel, support technique, audit de parc.
OPTIONS     :
  -t TYPE     Filtre par type DMI (numéro ou nom)
  -s MOT_CLÉ  Affiche une chaîne spécifique
  -q          Moins verbeux (pas d'en-têtes)
  -u          Données brutes
TYPES DMI COURANTS :
0             Informations BIOS
1             Informations système (modèle, SN, UUID)
2             Informations carte mère
4             Informations processeur
```

```

17          Barettes mémoire (slots RAM)
chassis      Type de châssis
processor    Processeur(s)
memory       Mémoire
bios         BIOS
EXAMPLES     :
$ sudo dmidecode -t 1
$ sudo dmidecode -t memory | grep -E "Size|Speed|Type|Manufacturer"
$ sudo dmidecode -t 4 | grep -E "Version|Speed|Core"
$ sudo dmidecode -s system-serial-number
$ sudo dmidecode -s bios-version

```

```
[USER] cpupower
```

```

DESCRIPTION : Gestion et monitoring des fonctionnalités de performance du CPU
POURQUOI    : Contrôler les fréquences CPU, le gouverneur (performance/powersave),
               l'idle state – optimiser la performance ou l'économie d'énergie.
CONTEXTE    : Optimisation de performances, économie d'énergie, benchmarking.
OPTIONS     :
  frequency-info      Infos sur la fréquence courante
  frequency-set -g GOUVERNEUR Définit le gouverneur (performance, powersave, ondemand)
  frequency-set -f FRÉQUENCE Définit une fréquence fixe
  idle-info           Informations sur les C-states
  idle-set -d N       Désactive le C-state N
  monitor            Monitore les fréquences en temps réel
GOUVERNEURS :
  performance        Fréquence maximale permanente
  powersave          Fréquence minimale permanente
  ondemand            Ajuste selon la charge (défaut)
  conservative        Comme ondemand mais plus progressif
  schedutil           Gouverneur basé sur le scheduler (moderne)
EXAMPLES          :
$ cpupower frequency-info
$ sudo cpupower frequency-set -g performance
$ sudo cpupower frequency-set -g powersave
$ cpupower monitor
$ cpupower idle-info

```

```
[SUDO] turbostat
```

```

DESCRIPTION : Affiche les statistiques de fréquence, température et consommation CPU
POURQUOI    : Monitoring détaillé des CPUs Intel : fréquences réelles, Turbo Boost,
               C-states, consommation RAPL, températures par cœur.
CONTEXTE    : Optimisation de performance, surveillance thermique, benchmarking CPU.
OPTIONS     :
  -i N           Intervalle de rafraîchissement en secondes
  -n N           Nombre d'itérations
  -s COLONNES    Colonnes à afficher
  -q            Mode silencieux (pas d'en-tête système)
  -c CPUS        CPUs spécifiques (ex: 0,2,4)
  --add COMPTEUR Ajoute un compteur MSR personnalisé
COLONNES CLÉS :
  Core/CPU       Numéro de cœur/CPU
  Avg_MHz        Fréquence moyenne réelle
  Busy%          Pourcentage d'activité
  Bzy_MHz        Fréquence quand occupé
  TSC_MHz        Fréquence TSC (référence)
  IPC            Instructions par cycle
  CoreTmp        Température du cœur
  PkgTmp         Température du paquet
  PkgWatt        Consommation du paquet (W)
  RAMWatt        Consommation RAM (W)
EXAMPLES       :
$ sudo turbostat
$ sudo turbostat -i 2
$ sudo turbostat --quiet sleep 10
$ sudo turbostat -s Core,Avg_MHz,Busy%,CoreTmp

```

C. RAPPORTS DE PERFORMANCE – SYSSTAT (sar, iostat, mpstat, pidstat...)

```
[USER] nfsiostat
```

```
DESCRIPTION : Statistiques d'entrées/sorties pour les montages NFS
POURQUOI    : Analyser les performances des montages NFS : latence, débit, opérations/s.
CONTEXTE    : Diagnostic de performances NFS, monitoring de serveurs de fichiers.
OPTIONS     :
  INTERVALLE : Rafraîchissement en secondes
  COUNT      : Nombre d'itérations
  -m         : Affiche les statistiques de montage
  -d         : Affiche les statistiques disque (device)
  -h         : Format lisible
  -o         : Affiche les statistiques d'opérations
  -p MONTAGE : Filtre sur ce montage
EXEMPLES    :
$ nfsiostat 2
$ nfsiostat -h 5 3
$ nfsiostat -m /mnt/nfs_partage
```

NOTE : Pour les statistiques système complètes (sar, mpstat, pidstat, iostat), installer le paquet sysstat :

```
$ sudo dnf install sysstat
$ sudo systemctl enable --now sysstat
```

COMMANDES SYSSTAT (si installées) :

```
.....
sar          : Rapport d'activité système (historique + temps réel)
              $ sar -u 1 5          (CPU toutes les 1s, 5 fois)
              $ sar -r 1 5          (mémoire)
              $ sar -d 1 5          (disques)
              $ sar -n DEV 1 5      (réseau)
              $ sar -b 1 5          (I/O globales)
              $ sar -f /var/log/sa/sa15 (rapport du 15 du mois)

mpstat       : Statistiques CPU multi-processeurs
              $ mpstat -P ALL 1 5   (toutes les CPUs)
              $ mpstat 1            (CPU globale en temps réel)

pidstat      : Statistiques par processus
              $ pidstat 1           (CPU par processus)
              $ pidstat -d 1        (I/O par processus)
              $ pidstat -r 1        (mémoire par processus)
              $ pidstat -u -r -d -p PID 1 (tout pour ce PID)

iostat       : Statistiques CPU et I/O disque
              $ iostat -x 1 5       (disques étendu)
              $ iostat -h 2         (format lisible)

sadf         : Affiche les données sysstat dans différents formats
              $ sadf -d -- -u       (CSV de l'utilisation CPU)
              $ sadf -j -- -r       (JSON de la mémoire)

cifsiosstat  : Statistiques I/O pour les montages CIFS/Samba
              $ cifsiosstat 2
```

D. PROFILAGE & ANALYSE DE PERFORMANCES

```
[USER] perf
```

```
DESCRIPTION : Outil de profilage de performance du noyau Linux (perf_events)
POURQUOI    : Profiler les performances à très bas niveau – compteurs matériels CPU,
              flamegraphs, analyse de cache misses, branches, IPC.
              L'outil de référence pour le profilage de performance Linux.
CONTEXTE    : Optimisation de code, débogage de performances, analyse de bottlenecks.
OPTIONS     :
  stat CMD   : Statistiques de compteurs pour une commande
  top        : Profiler en temps réel (comme top mais par fonction)
  record CMD : Enregistre un profil dans perf.data
  report     : Analyse perf.data et affiche le rapport
  annotate   : Annote le code source/assembleur avec les compteurs
  list       : Liste les événements disponibles
  script     : Converti perf.data en texte lisible
  trace CMD  : Trace les appels système (comme strace mais plus léger)
```

bench SOUS-COMMANDE	Benchmarks intégrés
mem CMD	Profilage des accès mémoire
lock CMD	Analyse des contentions de verrous
sched CMD	Analyse du scheduler
timechart	Génère un SVG de timeline d'activité
kvm	Statistiques KVM
diff	Compare deux fichiers perf.data

OPTIONS STAT :

-e ÉVÉNEMENT	Événement(s) à mesurer
-a	Tous les CPUs
-p PID	Processus existant
-r N	Répète N fois

ÉVÉNEMENTS COURANTS :

cycles	Cycles CPU
instructions	Instructions exécutées
cache-misses	Ratés de cache L1/L2/L3
cache-references	Références au cache
branch-misses	Prédictions de branchement ratées
page-faults	Fautes de page
context-switches	Changements de contexte
cpu-migrations	Migrations entre CPUs

EXEMPLES :

```
$ perf stat ./mon_programme
$ perf stat -e cycles,instructions,cache-misses ./prog
$ sudo perf top
$ sudo perf record -g ./mon_programme
$ perf report
$ perf annotate
$ perf list
$ perf stat -a sleep 5
$ sudo perf record -F 99 -a -g -- sleep 30
$ perf script | /usr/share/d3-flame-graph/flamegraph.pl > flame.svg
```

```
[USER] valgrind
```

DESCRIPTION : Framework d'instrumentation pour analyse mémoire et performances

POURQUOI : Détecter les fuites mémoire, les accès invalides, les races conditions, profiler l'utilisation du cache et des appels de fonctions.

CONTEXTE : Développement C/C++, débogage mémoire, optimisation de code.

OUTILS INTÉGRÉS :

memcheck (défaut)	Détection des erreurs mémoire
callgrind	Profilage des appels de fonctions et cache
cachegrind	Profilage du cache CPU et de la prédiction de branches
massif	Profilage de l'utilisation du tas (heap)
helgrind	Détection des races conditions (multi-thread)
drd	Détection des races conditions (alternative à helgrind)
exp-sgcheck	Vérification des accès pile et globaux

OPTIONS GLOBALES :

--tool=OUTIL	Sélectionne l'outil (défaut: memcheck)
--error-exitcode=N	Code de retour si erreurs détectées
--log-file=FICHIER	Écrit la sortie dans un fichier
--xml=yes --xml-file=FICHIER	Sortie XML
--quiet	Réduit la verbosité
--verbose	Mode verbeux
-v	Mode verbeux

OPTIONS MEMCHECK :

--leak-check=full	Rapport détaillé des fuites mémoire
--show-leak-kinds=all	Tous les types de fuites
--track-origins=yes	Montre l'origine des valeurs non-initialisées
--error-limit=no	Pas de limite sur le nombre d'erreurs
--suppressions=FICHIER	Fichier de suppressions (faux positifs)
--gen-suppressions=all	Génère des suppressions pour toutes les erreurs

OPTIONS CALLGRIND :

--callgrind-out-file=FICHIER	Fichier de sortie
--dump-instr=yes	Dump au niveau instruction
--collect-jumps=yes	Collecte les sauts conditionnels
--cache-sim=yes	Simulation du cache

Visualisation : callgrind_annotate ou KCachegrind

OPTIONS MASSIF :

--heap=yes	Profile le tas
--stacks=yes	Profile la pile aussi
--time-unit=ms	Unité de temps

Visualisation : ms_print ou massif-visualizer

EXEMPLES :

```
$ valgrind --leak-check=full --track-origins=yes ./monprog
$ valgrind --tool=callgrind ./monprog
$ callgrind_annotate callgrind.out.PID
$ valgrind --tool=massif ./monprog
$ ms_print massif.out.PID | head -30
$ valgrind --tool=helgrind ./prog_multithread
$ valgrind --error-exitcode=1 --leak-check=full ./tests
```

```
[USER] callgrind_annotate / callgrind_control
```

DESCRIPTION : callgrind_annotate : analyse et affiche les données callgrind
callgrind_control : contrôle un processus sous callgrind en cours

POURQUOI : Analyser les résultats de profilage callgrind, activer/désactiver la collecte à la volée.

CONTEXTE : Profilage de performances C/C++, analyse de hotspots.

EXEMPLES :

```
$ callgrind_annotate --auto=yes callgrind.out.12345
$ callgrind_annotate --inclusive=yes --sort=curB:100,curBs callgrind.out.12345
$ callgrind_control -i on 12345 (active la collecte pour le PID 12345)
$ callgrind_control -i off 12345 (désactive la collecte)
$ callgrind_control -d 12345 (dump les données)
```

```
[USER] gprof
```

DESCRIPTION : Profileur GNU – analyse le temps passé dans chaque fonction

POURQUOI : Profiler les programmes compilés avec -pg pour identifier les fonctions les plus gourmandes en temps CPU.

CONTEXTE : Optimisation de code C/C++/Fortran, développement.

PRÉREQUIS : Compiler avec : gcc -pg -g mon_programme.c -o mon_programme

OPTIONS :

-b	Pas de descriptions (sortie compacte)
-p	Affiche le profil plat (flat profile) uniquement
-q	Affiche le graphe d'appels uniquement
-A	Annote le code source
-l	Profil par ligne
-e FONCTION	Exclut cette fonction du graphe
-f FONCTION	Inclut seulement cette fonction

WORKFLOW :

1. gcc -pg -g prog.c -o prog
2. ./prog (génère gmon.out)
3. gprof prog gmon.out > rapport.txt

EXEMPLES :

```
$ gcc -pg -O2 monprog.c -o monprog
$ ./monprog
$ gprof monprog gmon.out | less
$ gprof -b monprog gmon.out | head -50
```

E. BENCHMARKING & TESTS DE PERFORMANCE

```
[USER] ab
```

DESCRIPTION : Apache Benchmark – outil de test de charge pour serveurs HTTP

POURQUOI : Mesurer rapidement les performances d'un serveur web : requêtes/s, latence, débit – sans configuration complexe.

CONTEXTE : Tests de performance web, validation avant déploiement, benchmarking.

OPTIONS :

-n N	Nombre total de requêtes
-c N	Nombre de requêtes concurrentes
-t N	Durée du test en secondes
-p FICHER	Fichier de données POST
-T CONTENT_TYPE	Content-Type pour POST
-H "En-tête: valeur"	En-tête HTTP personnalisé
-A USER:PASS	Authentification Basic
-k	Keep-Alive (connexions persistantes)
-e FICHER	Exporte les percentiles CSV
-g FICHER	Exporte les données gnuplot
-r	Ne pas s'arrêter sur les erreurs de socket
-s N	Timeout de socket en secondes
-l	Accepte les réponses de taille variable
-v N	Niveau de verbosité

MÉTRIQUES CLÉ :

Requests per second	Requêtes par seconde (débit)
Time per request	Latence moyenne
Transfer rate	Débit de transfert
Connection Times	Min/Mean/Max/Median des temps de connexion
Percentiles (50%, 66%, 75%...)	Distribution des latences

EXEMPLES :

```
$ ab -n 1000 -c 10 http://localhost/
$ ab -n 5000 -c 50 -k https://monsite.com/api/endpoint
$ ab -n 1000 -c 20 -p data.json -T application/json http://api.local/
$ ab -n 10000 -c 100 -e percentiles.csv http://serveur/
```

```
[SUDO] hdparm
```

DESCRIPTION : Configuration et benchmark des disques durs ATA/SATA/SSD

POURQUOI : Mesurer les performances de lecture d'un disque, configurer les paramètres ATA (DMA, cache, write cache, APM, AAM).

CONTEXTE : Benchmark disque, optimisation, configuration avancée ATA.

OPTIONS :

-t DISQUE	Test de lecture séquentielle (timing – bypass le cache OS)
-T DISQUE	Test de lecture depuis le cache du buffer
-I DISQUE	Infos complètes sur le disque (ATA IDENTIFY)
-d N	Active/désactive DMA (0/1)
-W N	Active/désactive le cache d'écriture (0/1)
-S N	Délai avant mise en veille du disque
-B N	Niveau APM (1=économie max, 254=perf max, 255=désactivé)
-M N	Niveau AAM (Automatic Acoustic Management, 0=off)
-a N	Taille de readahead en secteurs
-A N	Active/désactive le readahead lookahead
-g DISQUE	Affiche la géométrie du disque
-q	Mode silencieux

EXEMPLES :

```
$ sudo hdparm -t /dev/sda
$ sudo hdparm -T /dev/sda
$ sudo hdparm -I /dev/sda | grep -E "Model|Serial|Firmware|capacity"
$ sudo hdparm -W 0 /dev/sda (désactive le cache d'écriture – plus sûr)
$ sudo hdparm -B 127 /dev/sda (APM équilibré)
$ sudo hdparm -t --direct /dev/nvme0n1
```

F. RAPPORTS DE PANNES & DIAGNOSTIC – ABRT & COREDUMP

```
[USER] coredumpctl
```

DESCRIPTION : Gère et analyse les core dumps collectés par systemd-coredump

POURQUOI : Accéder facilement aux core dumps des programmes qui ont planté, les analyser avec gdb, les exporter.

CONTEXTE : Débogage de crashes, analyse post-mortem, support.

OPTIONS :

list	Liste tous les core dumps
info	Informations sur un core dump
dump	Exporte le core dump vers un fichier
debug	Ouvre le core dump dans gdb
gdb	Ouvre dans gdb (alias de debug)
-l	Dernier core dump
-n N	Dernier N core dumps
-S HORODATAGE	Depuis cette date
-U HORODATAGE	Jusqu'à cette date
-p PID	Filtre par PID
-u UTILISATEUR	Filtre par utilisateur
-e EXÉCUTABLE	Filtre par exécutable
--output=FICHIER	Fichier de sortie pour dump
--no-pager	Pas de pagination

EXEMPLES :

```
$ coredumpctl list
$ coredumpctl info -l
$ coredumpctl debug -l
$ coredumpctl dump -l --output=/tmp/core.dump
$ coredumpctl list nginx
$ coredumpctl info _EXE=/usr/bin/python3
```

```
[SUDO] abrt-d / abrt-cli
```

```
DESCRIPTION : ABRT (Automatic Bug Reporting Tool) – détection et rapport automatique de crashes
POURQUOI : Capturer automatiquement les crashes d'applications et noyau,
           générer des rapports de bugs, les envoyer à Bugzilla ou Red Hat.
CONTEXTE : Fedora/RHEL – reporting automatique de bugs au projet ou à Red Hat.
COMMANDES :
  abrt-d Daemon ABRT (service systemd)
  abrt-cli list Liste les problèmes détectés
  abrt-cli info ID Détail d'un problème
  abrt-cli report ID Soumet le rapport de bug
  abrt-cli remove ID Supprime un problème
  abrt-auto-reporting Configure le reporting automatique
SOUS-OUTILS ABRT :
  abrt-action-analyze-backtrace Analyse une backtrace
  abrt-action-analyze-c Analyse un crash C/C++
  abrt-action-analyze-python Analyse un crash Python
  abrt-action-analyze-oops Analyse un oops noyau
  abrt-action-analyze-vmcore Analyse un crash noyau complet
  abrt-action-generate-backtrace Génère une backtrace avec gdb
  abrt-dump-journal-oops Extrait les oops du journal systemd
  abrt-dump-journal-core Extrait les core dumps du journal
  abrt-dump-oops Extrait les oops du dmesg
  abrt-watch-log Surveille un fichier log pour les crashes
  abrt-harvest-vmcore Collecte les vmcore (crashdump noyau)
  abrt-harvest-pstoreoops Collecte les oops depuis pstore
  abrt-bodhi Cherche les mises à jour Fedora liées
EXEMPLES :
$ sudo systemctl status abrt-d
$ abrt-cli list
$ abrt-cli info /var/spool/abrt/ccpp-2024-01-15-12:00:00-1234
$ abrt-auto-reporting enabled
```

G. COLLECTE DE SUPPORT – SOS REPORT

```
[SUDO] sosreport / sos
```

```
DESCRIPTION : Collecte un rapport de support système complet (logs, configs, infos matérielles)
POURQUOI : Générer en une commande une archive complète de l'état du système
           pour le support Red Hat / Fedora / partenaires. Indispensable pour
           ouvrir un ticket de support.
CONTEXTE : Support Red Hat, diagnostic complexe, audit système complet.
OPTIONS :
  -l / --list-plugins Liste les plugins disponibles
  -e PLUGIN Active un plugin supplémentaire
  -n PLUGIN Désactive un plugin
  -o PLUGIN Collecte seulement ce plugin
  -a Active tous les plugins
  -k PLUGIN.OPTION=VALEUR Configure une option de plugin
  --batch Mode non-interactif
  --no-report Pas de rapport HTML
  --tmp-dir RÉPERTOIRE Répertoire temporaire
  --case-id ID Associe un numéro de cas support
  --upload Upload vers Red Hat (nécessite abonnement)
  -z / --compression-type TYPE Type de compression (auto, bzip2, gzip, xz)
  -p / --profile PROFIL Profil de collecte (boot, cluster, container...)
  collect Mode sos-collector (plusieurs nœuds simultanément)
  help PLUGIN Aide sur un plugin
PLUGINS COURANTS :
  system kernel hardware memory filesys logs
  networking services process storage security containers
EXEMPLES :
$ sudo sosreport
$ sudo sosreport --batch --case-id 12345678
$ sudo sos report -o networking,system --batch
$ sudo sos report -e memory -k memory.all=True --batch
$ sudo sos collect --nodes "node1,node2,node3"
$ sudo sos report -l
```

```
[USER] sos-collector
```

DESCRIPTION : Collecte des rapports sos sur plusieurs nœuds simultanément
 POURQUOI : Automatiser la collecte de diagnostics sur un cluster ou des groupes de serveurs en parallèle via SSH.
 CONTEXTE : Clusters, infrastructures multi-serveurs, support Red Hat cluster.
 OPTIONS :
 --nodes "N1,N2,..." Liste des nœuds
 --cluster-type TYPE Type de cluster (pacemaker, kubernetes, ocp...)
 --ssh-user USER Utilisateur SSH
 --ssh-port PORT Port SSH
 -j N Nombre de collectes parallèles
 EXEMPLES :
 \$ sudo sos-collector --nodes "srv1,srv2,srv3"
 \$ sudo sos-collector --cluster-type pacemaker

H. INFORMATIONS MATÉRIELLES COMPLÉMENTAIRES

[USER] cpuid-dump

DESCRIPTION : Dump brut des registres CPUID du processeur
 POURQUOI : Obtenir les informations brutes du processeur (CPUID) – flags, topology, cache, fonctionnalités avancées.
 CONTEXTE : Analyse matérielle avancée, développement bas-niveau, virtualisation.
 EXEMPLES :
 \$ cpuid-dump
 \$ cpuid-dump | less

[USER] cpuinfo

DESCRIPTION : Affiche les informations du processeur (peut être un alias de /proc/cpuinfo)
 POURQUOI : Récupérer les informations du CPU de façon lisible.
 CONTEXTE : Scripts, inventaire matériel.
 EXEMPLES :
 \$ cpuinfo
 \$ cat /proc/cpuinfo | grep "model name" | uniq

[USER] perfquery

DESCRIPTION : Interroge les compteurs de performance InfiniBand
 POURQUOI : Monitorer les performances et erreurs sur les réseaux InfiniBand/RDMA.
 CONTEXTE : Clusters HPC, stockage haute performance avec InfiniBand.
 OPTIONS :
 -x Affiche les compteurs étendus
 -X Affiche les compteurs de port étendus
 -a Tous les ports
 -r Remet les compteurs à zéro après lecture
 -c Affiche les compteurs de données combinés
 EXEMPLES :
 \$ perfquery
 \$ perfquery -x
 \$ perfquery -a

[SUDO] x86_energy_perf_policy

DESCRIPTION : Configure la politique de performance/énergie des processeurs Intel
 POURQUOI : Ajuster l'équilibre performance/consommation du processeur via les registres MSR EPB (Energy Performance Bias).
 CONTEXTE : Optimisation serveur, économie d'énergie, benchmarking.
 OPTIONS :
 performance Maximise la performance (ignore l'énergie)
 normal Équilibre (défaut)
 powersave Priorité à l'économie d'énergie
 -r Affiche la politique actuelle
 -c N Applique au CPU N
 EXEMPLES :
 \$ sudo x86_energy_perf_policy performance
 \$ sudo x86_energy_perf_policy -r
 \$ sudo x86_energy_perf_policy powersave